It Only Gets Worse: Revisiting DL-Based Vulnerability Detectors from a Practical Perspective

Yunqian Wang*
College of Intelligence and Computing
Tianjin University
Tianjin, China
2023244096@tju.edu.cn

Xiaohong Li*
College of Intelligence and Computing
Tianjin University
Tianjin, China
xiaohongli@tju.edu.cn

Ruitao Feng†
Faculty of Science and Engineering
Southern Cross University
Australia
ruitao.feng@scu.edu.au

Yao Zhang
College of Intelligence and Computing
Tianjin University
Tianjin, China
zzyy@tju.edu.cn

Yuekang Li
University of New South Wales
Australia
yuekang.li@unsw.edu.au

Zhiping Zhou
College of Intelligence and Computing
Tianjin University
Tianjin, China
zhou_zhiping@tju.edu.cn

***Abstract*—With the escalating threat of software vulnerabilities to the security of modern software systems, an increasing number of deep learning (DL) model-based vulnerability detectors have been developed for vulnerability detection. However, their practical reliability, consistency in usage, and adaptability across diverse software contexts remain unclear. This uncertainty may lead to unreliable detection results in practical applications, increased false positives and false negatives, and limited adaptability to newly emerged vulnerabilities. Conducting a large-scale and in-depth analysis of DL-based vulnerability detectors can help uncover critical factors influencing detection performance, improve the design and training of these models, and enhance their practical deployment in real-world scenarios.**

In this paper, we present VULTEGRA, a novel evaluation framework that, for the first time, conducts a multidimensional assessment comparing scratch-trained models and pre-trained-based models for vulnerability detection, while verifying key factors influencing detection performance. Our framework reveals that state-of-the-art (SOTA) detectors still suffer from low consistency, limited practical detection capabilities, and limited adaptability. Moreover, comparative results indicate that the increasingly favored pre-trained-based models are not universally superior to scratch-trained models; instead, they exhibit distinct strengths and application scenarios. Most importantly, our study highlights the limitations of relying solely on CWE-based classification and reveals a set of critical factors that significantly influence detection performance. Experimental validation shows that these factors have a substantial impact: modifying only any single factor led to recall improvements across all seven evaluated SOTA detectors, with six detectors also achieving higher F1 scores. Our findings provide deep insights into model behavior, highlighting the need to consider both vulnerability types and inherent code features to ensure practical applicability in real-world software environments.

I. INTRODUCTION

In recent years, the number of disclosed vulnerabilities in the Common Vulnerabilities and Exposures (CVE) [1] database has steadily increased, with a growing proportion classified as high-risk. This trend underscores the urgent need for robust detection tools to mitigate security risks and protect software systems from exploitation [2], particularly in practical deployment environments.

Traditional vulnerability detection relies on static analysis [3], [4], dynamic analysis [5], and symbolic execution [6]. While effective in certain cases, these techniques often struggle with scalability, high false positive rates, and unseen vulnerability patterns. DL-based vulnerability detection has emerged as a promising alternative [7], [8], broadly divided into scratch-trained and pre-trained-based models. Scratch-trained models are built from the ground up using vulnerability-specific datasets, enabling them to capture tailored patterns but requiring high-quality labeled data [9]. In contrast, pre-trained-based models are initialized with general knowledge from large-scale code corpora, allowing them to leverage broad representations for improved generalization [10]. Although large language models (LLMs) have advanced rapidly and shown strong performance across domains [11], existing studies [12], [13] suggest they still fall short of specialized SOTA detectors in vulnerability detection. Therefore, we focus solely on DL-based vulnerability detectors in this study.

Despite the success of scratch-trained and pre-trained-based models in vulnerability detection, our understanding of these detectors remains incomplete, particularly regarding their practical effectiveness and limitations in real-world software engineering. Chakraborty et al. [14] evaluated DL-based detectors on realistic datasets, but their study overlooked whether these detectors could consistently maintain high accuracy within their declared CWE [15] ranges or effectively detect newly emerging vulnerabilities. While Steenhoek et al. [16] studied CWE-related issues, their focus was on whether certain CWEs are easier to detect and whether to build per-CWE or general-purpose models, which differs from our goals. Moreover, neither study explored the differences between scratch-trained and pre-trained-based models. Most current research uses CWE

*These authors contributed equally to this work.
†Corresponding author.

as the sole criterion for sample selection [17], yet the CWE taxonomy does not reflect the intrinsic code characteristics of vulnerabilities. No existing work has examined whether hidden factors beyond CWE classification influence detection performance. Addressing these gaps is essential for developing more effective and adaptable detection models in practice.

Therefore, to better understand how different detection models behave in practical tasks and what factors influence their performance, we propose a novel framework VULTEGRA. For the first time, it conducts a multidimensional assessment of scratch-trained and pre-trained models, verifying key factors affecting detection performance. This framework enables comprehensive comparison, revealing differences in consistency, real-world effectiveness, and adaptability.

Leveraging this framework, we conducted an extensive evaluation involving four SOTA scratch-trained models and three SOTA pre-trained-based models for vulnerability detection. Our experimental results reveal several critical insights: (a) Existing detectors are still far from achieving stable and reliable results across their declared vulnerability scope, while the consistency among the seven detectors remains below 80%. (b) The actual detection capabilities of these detectors in the real world are limited, and further improvements are needed before these detectors can be used for real-world software vulnerability detection tasks. (c) While current detectors perform well on previously seen vulnerabilities, their adaptability to unknown vulnerabilities remains a critical gap. (d) Scratch-trained models and pre-trained-based models exhibit distinct strengths and application scenarios. (e) Modifying even a single key factor improved recall across all seven evaluated detectors, with six detectors also showing higher F1 scores. The largest improvements were a 35% increase in recall and a 13.8% increase in F1 score.

Our findings guide future research on DL-based vulnerability detection. By uncovering key factors influencing detection performance and evaluating the strengths and limitations of scratch-trained and pre-trained models, our work provides a foundation for designing more effective, adaptable detectors. Furthermore, the insights can inform better training data strategies, enhance generalization to unseen vulnerabilities, and support deployment of more reliable detectors in practice.

In summary, we make the following contributions:

- We propose a novel framework, VULTEGRA [18], which enables a multi-dimensional evaluation of SOTA DL-based vulnerability detectors. Utilizing this framework, we identify issues impacting the practical usage of the detectors, such as low detection consistency, limited real-world applicability, and poor adaptability.
- We present the first comparative study between scratch-trained and pre-trained models for vulnerability detection, revealing distinct strengths and weaknesses for varied scenarios. This highlights the importance of understanding model-specific characteristics and selecting the appropriate detector based on deployment requirements.
- We introduce a set of hidden factors overlooked by the current sample selection standard. Experimental validation shows these factors significantly influence detection performance, offering a practical pathway to enhance DL-based vulnerability detectors.

II. PRELIMINARY

A. DL-based Vulnerability Detectors

DL-based vulnerability detectors can be broadly divided into two categories based on their model construction methodology: scratch-trained models and pre-trained-based models.

Scratch-trained models refer to neural networks trained entirely from scratch using labeled vulnerability detection datasets [19]–[21]. They learn vulnerability-specific patterns directly from the provided data, making them highly adaptable to the characteristics of the training dataset [7]. However, their performance is heavily reliant on the quality, quantity, and diversity of the labeled data, and they may generalize poorly to unseen vulnerabilities [22].

In contrast, **pre-trained-based models** are built upon neural architectures that have been pre-trained on large code corpora, such as CodeBERT [23], GraphCodeBERT [24], or UniXcoder [25], then fine-tuned for vulnerability detection. These models leverage the rich representations from pre-training, enabling effective performance with limited labeled data.

```
1 char * char_enviroment_w32_execvp(char * data){
2     size_t dataLen = strlen(data);
3     char * environment = GETENV(ENV_VARIABLE);
4     if (environment != NULL) {
5         strncat(data+dataLen, environment, 100-dataLen-1);
6     }
7     return data;
8 }
```

Vulnerability Line

a) Vulnerability Detected by Scratch-trained Model but Missed by Pre-trained-based Model

```
1  static int php_stream_temp_cast(php_stream * stream,
       int castas, void **ret TSRMLS_DC ){
2      php_stream_temp_data *ts = (php_stream_temp_data *)stream->abstract;
       ......
32     membuf = php_stream_memory_get_buffer(ts->innerstream, &memsize);
33     file = php_stream_fopen_tmpfile();
34     php_stream_write(file, membuf, memsize);
35     pos = php_stream_tell(ts->innerstream);
36     php_stream_free_enclosed(ts->innerstream, PHP_STREAM_FREE_CLOSE);
37     ts->innerstream = file;
38     php_stream_encloses(stream, ts->innerstream);
39     php_stream_seek(ts->innerstream, pos, SEEK_SET);
       ......
42 }
```

b) Vulnerability Detected by Pre-trained-based Model but Missed by Scratch-trained Model

Fig. 1: Contrasting Performance of Scratch-Trained Model and Pre-trained-based Model

Despite increasing interest in pre-trained vulnerability detection models, their performance relative to scratch-trained ones across scenarios remains unclear. Figure 1 illustrates cases where each model succeeds while the other fails, showing context-dependent performance without a consistent winner. Thus, multidimensional evaluation is essential to understand their capabilities and applicability.

Motivation-1: Existing studies lack a comprehensive comparison of scratch-trained and pre-trained models for vulnerability detection. A multidimensional evaluation is essential to understand their practical detection capabilities and guide developing improved models.

B. Current Sample Selection Standard

Most vulnerability studies use the Common Weakness Enumeration (CWE), a hierarchical taxonomy classifying vulnerabilities by root causes and behavior patterns (e.g., buffer overflows, CWE-120). This facilitates systematic analysis, dataset annotation, and performance benchmarking [26].

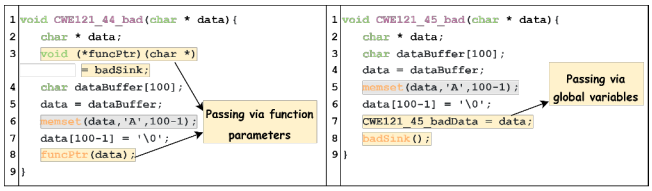


Fig. 2: Impact of Data Passing Mechanisms on Detecting Vulnerabilities in CWE-121

However, our early experiments exposed limitations in using CWE as the sole sample selection basis. Two samples from the same CWE category with nearly identical structures can yield different detection results. For instance, Figure 2 shows two CWE-121 samples differing only in data-passing methods: sample (a) is correctly detected, while sample (b) is misclassified. This indicates CWE-based classification may overlook subtle code-level variations, and finer-grained factors beyond CWE labels critically influence detection outcomes.

Motivation-2: CWE-based sample selection fails to capture subtle code-level differences. Since samples within the same CWE category can yield different results, exploring hidden factors beyond CWE labels is essential to enhance vulnerability detection accuracy.

III. Study on DL-based Vulnerability Detectors

A. Overview

Figure 3 provides an overview of the framework used to comprehensively evaluate and compare DL-based vulnerability detectors, focusing on scratch-trained and pre-trained-based models. The goal of this study is to develop a systematic understanding of the detection capabilities of state-of-the-art detectors, compare their strengths and weaknesses across multiple dimensions, and explore how hidden factors influence performance.

The following five research questions guide our study:

- **RQ1: How consistent are DL-based vulnerability detectors across different CWE categories?** This question assesses whether detectors maintain stable performance across their declared CWE categories.
- **RQ2: How effective are DL-based vulnerability detectors in detecting real-world software vulnerabilities?** This problem aims to evaluate the practical effectiveness of DL-based detectors by assessing their performance on real-world vulnerability datasets.

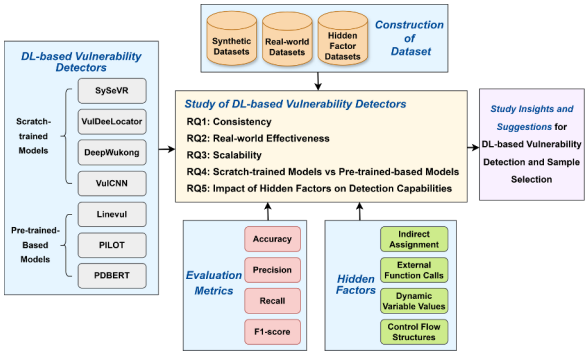


Fig. 3: Overview of the Study Framework

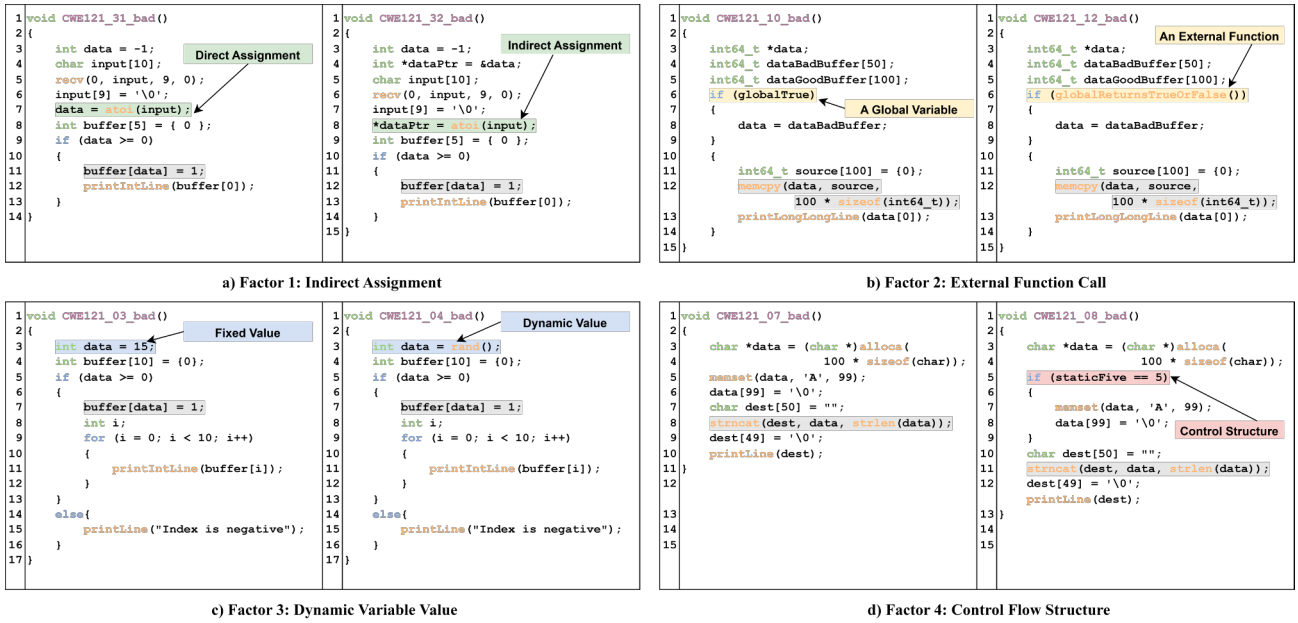
- **RQ3: Can DL-based vulnerability detectors effectively detect newly emerged vulnerabilities?** This question examines the adaptability and generalization of current detectors when facing previously unseen or newly emerged vulnerability instances.
- **RQ4: How do pre-trained-based models vs. scratch-trained models compare in vulnerability detection?** This question compares the detection capabilities of the two mainstream model paradigms and investigates whether pre-trained-based models consistently outperform their scratch-trained counterparts.
- **RQ5: Do hidden factors significantly affect the performance of DL-based vulnerability detectors?** This question verifies whether hidden factors other than CWE labels have a significant impact on the detection results.

By systematically answering these questions, our work aims to fill the current knowledge gaps in the understanding and practical application of DL-based vulnerability detection models. In particular, we conduct a first thorough comparative analysis of scratch-trained models and pre-trained-based models, revealing their respective advantages and suitable application scenarios. Furthermore, we identify and empirically verify a set of key influencing factors that substantially impact detection performance, offering new insights into how detection capabilities can be effectively enhanced.

B. Evaluation Metrics

The performance of the selected vulnerability detectors is evaluated using the following standard metrics:

- **Accuracy:** Measures the overall correctness of the detection model by calculating the proportion of correctly predicted vulnerability samples out of all the samples.
- **Precision:** Focuses on the correctness of positive predictions, indicating the proportion of true positive vulnerabilities out of all instances predicted as vulnerable.
- **Recall:** Measures the model's ability to detect actual vulnerabilities, indicating the proportion of true positive vulnerabilities out of all actual vulnerabilities.
- **F1 Score:** Provides a balanced evaluation between precision and recall, offering a single score to assess the model's overall detection performance.



a) Factor 1: Indirect Assignment

b) Factor 2: External Function Call

c) Factor 3: Dynamic Variable Value

d) Factor 4: Control Flow Structure

Fig. 4: Hidden Factors Affecting Detection Performance

These metrics are essential for comparing the effectiveness of different vulnerability detection models and understanding their reliability across various detection tasks.

C. Hidden Factors

In our evaluation of vulnerability detectors, we observed inconsistent detection results for samples within the same CWE category. Despite highly similar code structures, some were correctly identified, while others were misclassified. This prompted us to manually analyze over 500 samples, comparing differences between correct and incorrect classifications. Through this analysis, we identified four potential hidden factors that may impact detection performance: indirect assignment, external function calls, dynamic variable values, and control flow structures. Figure 4 illustrates these factors through comparative code samples.

Indirect assignment refers to cases where data is assigned to a variable through an intermediary operation instead of directly. Figure 4(a) shows two nearly identical code snippets, differing only in how data is passed. Indirect assignment can introduce subtle variations in data flow and memory access patterns, which some detectors may fail to capture accurately.

External function call involves using functions defined outside the immediate code context. Figure 4(b) compares two similar samples, where one includes a call to an external function while the other does not. External function calls can obscure the code's execution path and introduce uncertainty in data flow analysis, leading to inconsistent detection results.

Dynamic variable value refers to dynamically assigned values at runtime. Figure 4(c) presents two structurally similar code samples, differing only in the use of dynamic and static values. Dynamic values introduce variability and increase the complexity of control flow analysis, making it harder for detectors to accurately predict vulnerability states.

Control flow structure refers to the complexity of conditional and loop-based execution paths within a code sample. Figure 4(d) shows two code snippets that differ in control flow complexity. Complex control structures can obscure vulnerability patterns and confuse detectors.

D. Selected Detectors

The selected detectors comprise scratch-trained and pre-trained models, chosen for their SOTA performance in vulnerability detection and relevance in the field. Proposed by various research teams, these detectors cover diverse methodologies, enabling a comprehensive evaluation of their strengths and limitations. Table I summarizes key characteristics.

TABLE I: Studied DL-based Vulnerability Detectors

Type	Detector	Architecture	Detection Level
Scratch-trained	SySeVR [27]	RNN	Function
	VulDeeLocator [20]	Bi-LSTM	Function
	DeepWuKong [28]	GCN	Function
	VulCNN [29]	CNN	Function
Pre-trained-based	LineVul [30]	Transformer	Function/Line
	PILOT [31]	Transformer	Function
	PDBERT [32]	Transformer	Function

These detectors are considered SOTA due to their innovative architectures, ability to handle diverse vulnerability patterns, and consistent high performance across multiple CWE categories. They have been widely recognized by multiple research studies as leading approaches in the field of software vulnerability detection [33]–[38]. In this study, we reproduced these detectors using their original datasets and code to ensure

fidelity to their reported performance and to facilitate a fair and accurate comparison.

E. Dataset

The dataset comprises three components: real-world, synthetic, and hidden-factor data. Real-world data are sourced from public datasets, including NVD [39], Devign [40], Reveal [41], and Fan [42], providing diverse vulnerability cases. These datasets underwent preprocessing, including deduplication, formatting, and removal of irrelevant samples, to ensure consistency and reduce noise during model training. Synthetic data, sourced from the SARD dataset [43], are classified by CWE taxonomy and supplement real-world data with explicit CWE labels to meet experimental needs.

The hidden-factor dataset is designed to examine limitations of CWE-based classification and identifies factors affecting vulnerability detection. It comprises five subsets, created by modifying 100 real-world vulnerable and 100 non-vulnerable samples based on four factors. The first subset includes unmodified samples as a baseline. The second subset removes indirect memory allocations without altering sample structure to test detector sensitivity to memory handling. The third subset eliminates external function calls while retaining core logic to assess the impact of external dependencies. The fourth subset replaces dynamic variable values with static ones to eliminate runtime variability. The fifth subset simplifies control flow structures to evaluate detector sensitivity to logical complexity. This dataset supports a detailed analysis of factors influencing detection outcomes beyond CWE classification.

The dataset contains 31,673 vulnerability samples and 208,451 non-vulnerability samples, with 80% sourced from real-world scenarios. This dataset supports a balanced and realistic evaluation environment, improving the reliability and generalizability of experimental results. Tailored subsets address specific research questions (RQs). For RQ1, the dataset includes 28 CWE categories, with 4,591 vulnerability and 6,732 non-vulnerability samples. For RQ2, it comprises 27,418 vulnerability and 203,317 non-vulnerability samples. For RQ3, it includes 20 vulnerability samples across 12 CWE categories and four open-source projects, each with over 8,000 forks. RQ4 analyzes results from RQ1, RQ2, and RQ3, without a separate statistical range. For RQ5, the dataset consists of 500 vulnerability and 500 non-vulnerability samples, divided into five sub-datasets. These dataset scales ensure sufficient and representative samples for robust conclusions.

IV. Experiments and Results

A. RQ1: Consistency across Different CWE Categories

To assess the consistency of vulnerability detectors, we evaluated their accuracy across specific CWE types. A detector was considered consistent for a CWE if accuracy exceeded 90%. Performance evaluation was based on each model's claimed vulnerability type range. All detectors were reproduced using original datasets and code for fair comparison.

CWE types were ranked by prevalence using CWE TOP25 [44]. For detectors claiming over 10 types, accuracy was evaluated for the top 10 prevalent CWEs within their declared range. For those claiming less than 10 types, all explicitly stated types were evaluated. For models without claims, evaluation was restricted to the top 10 common CWEs. When insufficient samples existed for the top 10 CWEs, subsequent CWEs were evaluated until sufficient samples were obtained.

By structuring the experiments in this way, we ensured each detector's consistency was assessed under real-world conditions and tested performance on common and declared-scope vulnerabilities.

Result: The experimental results in Table II revealed that most detectors struggled with consistent performance on claimed vulnerabilities. SySeVR, covering a broad range, consistently exceeded 90% accuracy for only 20% of CWEs. Although LineVul reached 70%, it was still far from achieving stable and reliable results.

The performance also varied when it came to detecting specific vulnerability types. All six models failed to surpass 90% accuracy for CWE-78, highlighting a common limitation: current vulnerability detection models lack stable detection capabilities for certain types of vulnerabilities. These failures indicate a significant capability gap, necessitating enhancements in robustness and generalization to handle diverse types before reliable real-world application.

Observation-1: DL-based vulnerability detectors exhibit low consistency across declared vulnerability types, with significant performance gaps, especially for complex vulnerabilities.

B. RQ2: Effectiveness on Real-world Vulnerabilities

The experiment aimed to evaluate the real-world effectiveness of DL-based vulnerability detectors. Despite high accuracy on synthetic datasets, their real-world performance remains uncertain. This experiment assessed whether existing DL-based detectors effectively detect vulnerabilities in practical scenarios.

Seven state-of-the-art DL-based detectors were evaluated using the real-world dataset. Detection performance was measured with Precision, Recall, and F1-score metrics. Precision indicates the ability to minimize false positives, Recall indicates capturing true vulnerabilities, and F1-score balances Precision and Recall.

Result: Table III shows the experimental results. Overall, the seven DL-based detectors exhibited suboptimal performance. Among the evaluated detectors, PDBert achieved the highest precision at 0.59 and the highest f1-score at 0.56, indicating a better balance between true positive identification and false positive reduction. In terms of Recall, LineVul achieved the highest recall at 0.53, suggesting superior vulnerability detection at the cost of lower precision and f1-score. The gap between precision and recall across different models reflects the trade-off between capturing more vulnerabilities and minimizing incorrect classifications.

TABLE II: Performance of DL-based Vulnerability Detectors on Different CWEs. Values greater than or equal to 0.9, indicating satisfactory performance, are highlighted in cyan, while values below 0.9 are highlighted in red. A dash (–) indicates that the corresponding CWE is not within the evaluation scope and thus has no data available.

CWE	SySeVR	VulDeeLocator	DeepWuKong	VulCNN	LineVul	PILOT	PDBert
CWE-17	–	–	–	–	0.862	–	–
CWE-20	0.790	0.761	0.678	–	–	0.900	0.929
CWE-22	0.781	0.827	0.746	–	0.715	0.906	0.911
CWE-78	0.375	0.618	0.625	0.850	–	0.732	0.805
CWE-79	–	–	–	–	–	0.500	0.500
CWE-89	0.562	0.605	–	–	–	0.913	0.928
CWE-119	0.928	0.918	0.916	–	–	0.819	0.836
CWE-121	0.872	0.909	–	0.823	–	–	–
CWE-122	–	–	–	0.601	–	–	–
CWE-124	–	–	–	0.943	–	–	–
CWE-125	–	–	0.827	–	–	0.794	0.752
CWE-126	–	–	–	0.717	–	–	–
CWE-134	–	–	–	1.000	–	–	–
CWE-190	0.932	0.946	0.951	–	–	–	–
CWE-254	–	–	0.763	–	1.000	–	–
CWE-269	–	–	–	–	0.841	–	–
CWE-284	–	–	–	–	1.000	–	–
CWE-285	–	–	–	–	1.000	–	–
CWE-311	–	–	–	–	1.000	–	–
CWE-358	–	–	–	–	1.000	–	–
CWE-399	–	–	0.853	–	–	–	–
CWE-400	0.813	0.856	0.869	–	–	–	–
CWE-415	–	–	–	–	1.000	–	–
CWE-416	0.604	0.647	–	–	–	0.662	0.623
CWE-476	0.485	0.592	–	–	–	–	–
CWE-617	–	–	–	–	1.000	–	–
CWE-787	–	–	0.902	–	–	0.863	0.907
CWE-862	–	–	–	–	–	0.000	0.000

Notably, both PDBert and PILOT, which are pre-trained-based models, achieved results that closely align with their originally claimed performance, likely due to their strong representation capabilities. Their ability to generalize across diverse codebases stems from contextual semantics and syntax learned from large-scale code corpora. Furthermore, benefiting from transfer learning, they maintain high detection efficacy without extensive domain-specific fine-tuning. The stable performance of PDBert and PILOT underscores the advantage of integrating pre-trained contextual embeddings into vulnerability detection tasks, making them more resilient to domain shifts.

Observation-2: Existing DL-based vulnerability detectors struggle with real-world code complexity, achieving limited precision and recall. It is possible that pre-trained-based models can provide a scalable and transferable basis for real-world vulnerability detection across different codebases, compared to scratch-trained models.

C. RQ3: Adaptability to Newly Emerged Vulnerabilities

To assess the adaptability of the selected vulnerability detectors to newly disclosed vulnerabilities, we collected 20 CVEs disclosed in 2024. All were selected from actively maintained open-source projects with over 8,000 forks, representing real-world, high-impact security threats. This selection ensures evaluation on vulnerabilities with practical significance.

The seven selected vulnerability detectors were tested on this set of 20 new CVE vulnerabilities. Instead of using

TABLE III: Evaluation Results for real-world effectiveness of DL-based Vulnerability Detectors. The report column refers to the indicator data given by the detector authors in their papers. A dash (-) indicates that the paper did not present the data. An asterisk (*) indicates that the result was tested on a real-world dataset.

Detector	Precision		Recall		F1-score	
	Report	Test	Report	Test	Report	Test
SySeVR	0.90	0.32	-	0.47	0.92	0.38
VulDeeLocator	0.81*	0.35	-	0.52	0.80*	0.42
DeepWukong	-	0.43	-	0.52	0.90*	0.47
VulCNN	-	0.41	0.91	0.49	-	0.45
LineVul	0.97*	0.56	0.86*	**0.53**	0.91*	0.54
PILOT	0.54*	0.57	0.55*	0.52	0.55*	0.54
PDBert	-	**0.59**	-	0.53	0.59*	**0.56**

traditional metrics like precision and recall, we focused on recording the number of vulnerabilities correctly identified by each detector out of the 20 tested. For each detector, we reported the number of successfully detected vulnerabilities to evaluate how well they generalize to previously unseen vulnerabilities.

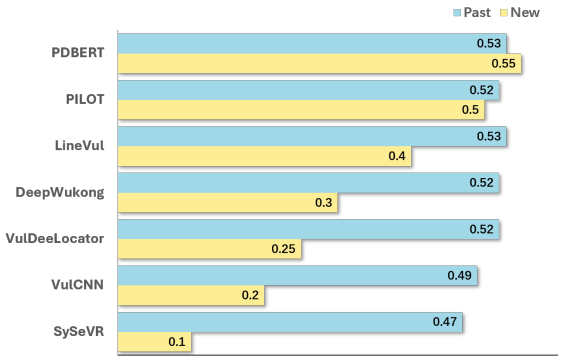


Fig. 5: Evaluation Results for Adaptability

Result: Figure 5 shows the experimental results, revealing that all detectors performed poorly on the 2024 CVEs. Their accuracy was notably lower than on training or known vulnerability datasets.

The SySeVR detector exhibited the lowest detection reliability, with only 10% accuracy, highlighting its limited generalization to new vulnerabilities. This may result from its narrow training data coverage. In contrast, PDBert and PILOT achieved relatively higher accuracy, with PDBert reaching 55%. Though still inadequate for deployment, their performance remained close to that on older datasets, unlike other detectors. This stability likely stems from rich semantic and syntactic representations learned during pre-training, allowing better generalization to unseen code structures and vulnerability types.

Observation-3: SOTA detectors failed to scale to new vulnerabilities, while PDBert and PILOT retained relatively stable performance, highlighting the superior generalization of pre-trained-based models despite their limitations on unseen threat patterns, compared to scratch-trained models.

D. RQ4: Scratch-trained Models vs. Pre-trained-based Models

The objective of this experiment is to conduct a comprehensive comparison between scratch-trained models and pre-trained-based models for vulnerability detection from the perspectives of consistency, real-world effectiveness, and adaptability. While both types of detectors have demonstrated promising performance in vulnerability detection, it remains unclear how their detection capabilities differ across various vulnerability types and practical scenarios. This experiment aims to identify their respective strengths and weaknesses, providing insights into the types of detection tasks each model type is more suited for. To achieve this, we evaluated the performance of four state-of-the-art scratch-trained models and three state-of-the-art pre-trained-based models.

Result: The experimental results reveal that scratch-trained and pre-trained models have distinct strengths and application scenarios.

According to Table II, pre-trained models performed significantly better on CWE-20 and CWE-89, where semantic context is critical. This advantage can be attributed to their architecture leveraging code language models, which capture rich contextual dependencies and token-level semantics. Conversely, scratch-trained models outperformed on CWE-119, which involves structured code patterns and memory handling. These findings suggest that pre-trained models excel in context-sensitive scenarios, while scratch-trained models are more effective for pattern-based vulnerabilities.

As shown in Table III, pre-trained-based models achieved higher precision and F1-scores than scratch-trained models, indicating better false positive control and balanced performance. Moreover, pre-trained-based models demonstrated better adaptability to newly introduced or previously unseen vulnerabilities, as evidenced by Figure 5. This suggests that the broader knowledge base and deeper contextual understanding of pre-trained-based models enable them to generalize more effectively to novel vulnerability patterns.

Observation-4: Pre-trained-based models excel in detecting complex, semantic vulnerabilities and adapting to new patterns, while scratch-trained models perform better on structured vulnerabilities. This suggests that combining both model types could enhance overall detection performance and adaptability in real-world scenarios.

E. RQ5: Impact of Hidden Factors on Detection Capabilities

To further explore the impact of hidden factors on detection performance, we designed a series of controlled experiments. Our experimental setup involved creating five datasets by manually modifying real-world samples to isolate four hidden factors, namely indirect assignment, external function calls, dynamic variable values, and control flow structures, which were identified through a detailed analysis of over 500 samples (see § III-E for the details of used data). These modifications were performed by subtracting specific elements from the impacting lines of code based on each factor, thus ensuring controlled variations. We tested seven different vulnerability detectors on this hidden factor dataset, conducting five independent tests for each configuration. Accuracy, precision, recall, and F1-score were used to measure the detectors' performance across different scenarios.

Result: The experimental results are presented in Table IV. Modifying any single key factor increased recall across all seven SOTA detectors. Moreover, six of these detectors also showed improved F1-scores. LineVul showed the greatest improvement, with recall increasing by 35% and F1-score by 13.8%. These results suggest that hidden factors hindered

TABLE IV: The Impact of Key Factors on Detection Performance. ↑represents the increase in the metric compared to that in the No-factor dataset, while ↓indicates the decrease in the metric relative to that in the No-factor dataset. No factor refers to the original sample without modification. Factor X means factor X is eliminated from the sample.

Detector	Factor	Acc.	Prec.	Rec.	F1
SySeVR	No-factor	0.300	0.308	0.320	0.314
	Factor-1	↑0.335	↑0.337	↑0.340	↑0.338
	Factor-2	↑0.360	↑0.357	↑0.350	↑0.353
	Factor-3	↑0.365	↑0.358	↑0.340	↑0.349
	Factor-4	↑0.340	↑0.337	↑0.330	↑0.333
VulDeeLocator	No-factor	0.330	0.327	0.340	0.333
	Factor-1	↑0.355	↑0.359	↑0.370	↑0.364
	Factor-2	↑0.390	↑0.396	↑0.420	↑0.408
	Factor-3	↑0.385	↑0.388	↑0.400	↑0.394
	Factor-4	↑0.370	↑0.375	↑0.390	↑0.382
DeepWukong	No-factor	0.375	0.381	0.400	0.390
	Factor-1	↑0.410	↑0.412	↑0.420	↑0.416
	Factor-2	↑0.440	↑0.442	↑0.460	↑0.451
	Factor-3	↑0.470	↑0.467	↑0.430	↑0.448
	Factor-4	↑0.425	↑0.426	↑0.420	↑0.423
VulCNN	No-factor	0.350	0.353	0.360	0.356
	Factor-1	↑0.375	↑0.379	↑0.390	↑0.384
	Factor-2	↑0.415	↑0.416	↑0.420	↑0.418
	Factor-3	↑0.400	↑0.402	↑0.410	↑0.406
	Factor-4	↑0.390	↑0.388	↑0.380	↑0.384
LineVul	No-factor	0.505	0.667	0.020	0.575
	Factor-1	↑0.530	↓0.607	↑0.170	↓0.566
	Factor-2	↑0.640	↑0.804	↑0.370	↑0.713
	Factor-3	↑0.585	↑0.793	↑0.230	↑0.673
	Factor-4	↓0.400	↓0.400	↑0.280	↓0.400
PILOT	No-factor	0.445	0.443	0.430	0.436
	Factor-1	↑0.480	↑0.480	↑0.470	↑0.470
	Factor-2	↑0.530	↑0.532	↑0.500	↑0.520
	Factor-3	↑0.485	↑0.485	↑0.470	↑0.477
	Factor-4	↑0.500	↑0.500	↑0.450	↑0.474
PDBert	No-factor	0.460	0.459	0.450	0.454
	Factor-1	↑0.485	↑0.485	↑0.470	↑0.477
	Factor-2	↑0.540	↑0.542	↑0.520	↑0.531
	Factor-3	↑0.510	↑0.510	↑0.490	↑0.500
	Factor-4	↑0.517	↑0.515	↑0.460	↑0.486

accurate detection, and their removal enhanced true positive identification.

External function calls and dynamic variable values had stronger effects, while removing indirect assignments and complex control flows yielded mixed outcomes. For example, LineVul's precision and F1-score declined when tested without indirect assignments or with simplified control flows. This may stem from its reliance on line-level structural cues, which are obscured when such features are removed. Thus, simplification may improve recall but challenge precision, revealing a complex interplay between hidden factors and detection strategies.

Overall, these findings highlight the need to account for hidden factors, as they substantially affect detector performance in real-world scenarios.

Observation-5: Hidden factors such as external function calls, dynamic variable values, indirect assignment, and control flow structures significantly influence detection performance, with the highest increase in recall being 35% and the highest F1-score increase being 13.8%. Incorporating these fine-grained code characteristics into classification and detection models could enhance consistency and reliability in real-world scenarios.

V. Discussion & Threat to Validity

A. Discussion

Our study highlights challenges and opportunities for DL-based vulnerability detectors, which show promise but require refinement to enhance reliability and adaptability.

1) Consistency and Adaptability of Existing Detectors: One of the most prominent challenges we observed is the inconsistency and limited adaptability of existing detectors. While effective for specific vulnerability types, they fail to generalize across diverse vulnerabilities in real-world systems. These models are not yet ready for direct deployment in practical scenarios without further optimization, particularly in terms of the ability to handle large-scale, complex software projects.

2) Reasonable Use of Two Models to Improve Detection Capabilities: Task-specific model selection is crucial to improve detection performance, as scratch-trained models excel at structured pattern-based vulnerabilities, while pre-trained models handle complex semantic vulnerabilities. Combining both enhances accuracy and adaptability.

3) Importance of Sample Code Features: Within a single CWE category, sample feature variations may cause different detection outcomes. These impact detection model performance, necessitating evaluation based on both vulnerability categorization and sample-specific features.

4) Future Work: Building on these insights, future research should prioritize hybrid techniques integrating scratch-trained and pre-trained models to address diverse real-world vulnerabilities, potentially through ensemble frameworks. Concurrently, enhanced datasets must incorporate hidden code features to improve training effectiveness and enable fairer capability assessment.

B. Threat to Validity

Despite our efforts to ensure a comprehensive and realistic evaluation, several potential threats to validity may influence our findings.

1) Data Representativeness: To closely approximate real-world scenarios, we constructed datasets with rich and diverse samples, most of which are sourced from real-world projects and publicly disclosed vulnerabilities. However, these may not fully capture production environment complexity, including programming style and architecture variations.

2) Factor Generalizability: To ensure generality and representativeness, we manually analyzed over 500 vulnerability samples, extracting four key factors influencing detection performance. Although these factors showed significant impact across multiple detectors and datasets, they may not generalize to other contexts where different programming languages, vulnerability types, or development paradigms introduce additional factors.

VI. Related Work

Throughout the course of our research, we have reviewed several relevant studies in the field of vulnerability detection. In this section, we provide a summary of the remaining ones and compare them with our contribution.

Vulnerability Detection Techniques: Vulnerability detection has evolved from traditional static analysis to deep learning-based approaches. Static analysis [45]–[47] identifies potential vulnerabilities by examining source code without execution but suffers from limited adaptability, high false positives, and poor handling of unknown patterns [48]–[50]. Deep learning techniques, in contrast, offer data-driven alternatives, divided into scratch-trained models, which learn vulnerability-specific patterns from labeled datasets using architectures like CNNs or hybrid networks [51]–[55], and pre-trained models, which first learn general code semantics from large corpora and are then fine-tuned for vulnerability detection [56]–[59]. These methods improve generalization, adaptability, and detection accuracy, marking a major advancement in automated vulnerability detection.

Empirical Analysis of Vulnerability Detection: Prior studies highlight limitations of DL-based vulnerability detection models in real-world settings [12], [16], [35], [60]. Despite success in controlled environments, these models often struggle with low accuracy and generalization, relying on superficial patterns. Chakraborty et al. [14] show performance drops in practice, causing high false positives and negatives, while Ding et al. [61] report that Code LMs, even with fine-tuning, fail to meet real-world demands. These challenges motivate our work, which examines multidimensional model capabilities and the impact of subtle code feature variations. Our work builds on these insights by focusing on the multi-dimensional capabilities of existing models and exploring the impact of subtle variations in code features.

VII. Conclusion

This work presents an in-depth analysis of the current state of DL-based vulnerability detectors. Our study reveals that current detectors suffer from low consistency, limited real-world effectiveness, and poor adaptability to newly emerged vulnerabilities. Our comparative analysis showed that pre-trained-based models, while generally more robust, are not universally superior, and both scratch-trained models and pre-trained-based models exhibit distinct strengths depending on the application scenario. Most importantly, our study identified a set of key factors beyond CWE classification that significantly influence detection performance. These findings offer practical guidance for developing more robust, generalizable, and effective vulnerability detection models.

Acknowledgment

This work was supported by the National Natural Science Foundation of China (Key Program, Grant No. 62332005).

References

[1] MITRE Corporation, "Common Vulnerabilities and Exposures (CVE)," https://cve.mitre.org, accessed: March 12, 2025.

[2] M. Shahzad, M. Z. Shafiq, and A. X. Liu, "Large scale characterization of software vulnerability life cycles," *IEEE Transactions on Dependable and Secure Computing*, vol. 17, no. 4, pp. 730–744, 2019.

[3] J. Walden and M. Doyle, "Savi: Static-analysis vulnerability indicator," *IEEE Security & Privacy*, vol. 10, no. 3, pp. 32–39, 2012.

[4] B. Chess and G. McGraw, "Static analysis for security," *IEEE security & privacy*, vol. 2, no. 6, pp. 76–79, 2004.

[5] J. Vadayath, M. Eckert, K. Zeng, N. Weideman, G. P. Menon, Y. Fratantonio, D. Balzarotti, A. Doupé, T. Bao, R. Wang *et al.*, "Arbiter: Bridging the static and dynamic divide in vulnerability discovery on binary programs," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 413–430.

[6] F. Busse, P. Gharat, C. Cadar, and A. F. Donaldson, "Combining static analysis error traces with dynamic symbolic execution (experience paper)," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022, pp. 568–579.

[7] G. Lin, S. Wen, Q.-L. Han, J. Zhang, and Y. Xiang, "Software vulnerability detection using deep neural networks: a survey," *Proceedings of the IEEE*, vol. 108, no. 10, pp. 1825–1848, 2020.

[8] J. Qiu, J. Zhang, W. Luo, L. Pan, S. Nepal, and Y. Xiang, "A survey of android malware detection with deep neural models," *ACM Computing Surveys (CSUR)*, vol. 53, no. 6, pp. 1–36, 2020.

[9] S. Wang, T. Liu, J. Nam, and L. Tan, "Deep semantic feature learning for software defect prediction," *IEEE Transactions on Software Engineering*, vol. 46, no. 12, pp. 1267–1293, 2018.

[10] X.-C. Wen, C. Gao, S. Gao, Y. Xiao, and M. R. Lyu, "Scale: Constructing structured natural language comment trees for software vulnerability detection," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 235–247.

[11] X. Zhu, W. Zhou, Q.-L. Han, W. Ma, S. Wen, and Y. Xiang, "When software security meets large language models: A survey," *IEEE/CAA Journal of Automatica Sinica*, vol. 12, no. 2, pp. 317–334, 2025.

[12] X. Yin, C. Ni, and S. Wang, "Multitask-based evaluation of open-source llm on software vulnerability," *IEEE Transactions on Software Engineering*, 2024.

[13] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "Llms cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 862–880.

[14] P. Chakraborty, K. K. Arumugam, M. Alfadel, M. Nagappan, and S. McIntosh, "Revisiting the performance of deep learning-based vulnerability detection on realistic datasets," *IEEE Transactions on Software Engineering*, 2024.

[15] MITRE, "Common Weakness Enumeration (CWE)," https://cwe.mitre.org/.

[16] B. Steenhoek, M. M. Rahman, R. Jiles, and W. Le, "An empirical study of deep learning models for vulnerability detection," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2237–2248.

[17] S. Pan, L. Bao, X. Xia, D. Lo, and S. Li, "Fine-grained commit-level vulnerability type prediction by cwe tree structure," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 957–969.

[18] VulTegra, "Vultegra home," https://sites.google.com/view/vultegra.

[19] H. Wang, G. Ye, Z. Tang, S. H. Tan, S. Huang, D. Fang, Y. Feng, L. Bian, and Z. Wang, "Combining graph-based learning with automated data collection for code vulnerability detection," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 1943–1958, 2020.

[20] Z. Li, D. Zou, S. Xu, Z. Chen, Y. Zhu, and H. Jin, "Vuldeelocator: a deep learning-based fine-grained vulnerability detector," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 4, pp. 2821–2837, 2021.

[21] Y. Li, S. Wang, and T. N. Nguyen, "Vulnerability detection with fine-grained interpretations," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 292–303.

[22] G. Bhandari, A. Naseer, and L. Moonen, "Cvefixes: automated collection of vulnerabilities and their fixes from open-source software," in *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering*, 2021, pp. 30–39.

[23] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang *et al.*, "Codebert: A pre-trained model for programming and natural languages," *arXiv preprint arXiv:2002.08155*, 2020.

[24] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu *et al.*, "Graphcodebert: Pre-training code representations with data flow," *arXiv preprint arXiv:2009.08366*, 2020.

[25] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "Unixcoder: Unified cross-modal pre-training for code representation," *arXiv preprint arXiv:2203.03850*, 2022.

[26] S. A. Atiiq, C. Gehrmann, K. Dahlén, and K. Khalil, "From generalist to specialist: Exploring cwe-specific vulnerability detection," *arXiv preprint arXiv:2408.02329*, 2024.

[27] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, "Sysevr: A framework for using deep learning to detect software vulnerabilities," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 4, pp. 2244–2258, 2021.

[28] X. Cheng, H. Wang, J. Hua, G. Xu, and Y. Sui, "Deepwukong: Statically detecting software vulnerabilities using deep graph neural network," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 30, no. 3, pp. 1–33, 2021.

[29] Y. Wu, D. Zou, S. Dou, W. Yang, D. Xu, and H. Jin, "Vulcnn: An image-inspired scalable vulnerability detection system," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 2365–2376.

[30] M. Fu and C. Tantithamthavorn, "Linevul: A transformer-based line-level vulnerability prediction," in *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022, pp. 608–620.

[31] X.-C. Wen, X. Wang, C. Gao, S. Wang, Y. Liu, and Z. Gu, "When less is enough: Positive and unlabeled learning model for vulnerability detection," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 345–357.

[32] Z. Liu, Z. Tang, J. Zhang, X. Xia, and X. Yang, "Pre-training by predicting program dependencies for vulnerability analysis tasks," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.

[33] Y. Nong, R. Fang, G. Yi, K. Zhao, X. Luo, F. Chen, and H. Cai, "Vgx: Large-scale sample generation for boosting learning-based software vulnerability analyses," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.

[34] F. Qiu, Z. Liu, X. Hu, X. Xia, G. Chen, and X. Wang, "Vulnerability detection via multiple-graph-based code representation," *IEEE Transactions on Software Engineering*, 2024.

[35] X.-C. Wen, X. Wang, Y. Chen, R. Hu, D. Lo, and C. Gao, "Vuleval: Towards repository-level evaluation of software vulnerability detection," *arXiv preprint arXiv:2404.15596*, 2024.

[36] X. Yang, S. Wang, Y. Li, and S. Wang, "Does data sampling improve deep learning-based vulnerability detection? yeas! and nays!" in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2287–2298.

[37] Y. Hu, S. Wang, W. Li, J. Peng, Y. Wu, D. Zou, and H. Jin, "Interpreters for gnn-based vulnerability detection: Are we there yet?" in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 1407–1419.

[38] X. Cheng, G. Zhang, H. Wang, and Y. Sui, "Path-sensitive code embedding via contrastive learning for software vulnerability detection," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022, pp. 519–531.

[39] National Institute of Standards and Technology (NIST), "National Vulnerability Database (NVD)," https://nvd.nist.gov/.

[40] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," *Advances in neural information processing systems*, vol. 32, 2019.

[41] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?" *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3280–3296, 2021.

[42] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "Ac/c++ code vulnerability dataset with code changes and cve summaries," in *Proceedings of the 17th international conference on mining software repositories*, 2020, pp. 508–512.

[43] National Institute of Standards and Technology (NIST), "SARD Dataset," https://samate.nist.gov/SRD/index.php.

[44] MITRE, "CWE TOP 25," https://cwe.mitre.org/top25/archive/2024/2024_cwe_top25.html.

[45] Cppcheck team, "Cppcheck," https://cppcheck.sourceforge.io/, 2021.

[46] SonarSource, "SonarQube - Continuous Code Quality," https://www.sonarsource.com/products/sonarqube/, 2021.

[47] Jeremy Long and OWASP, "OWASP Dependency Check," https://owasp.org/www-project-dependency-check/, 2021.

[48] M. Nachtigall, M. Schlichtig, and E. Bodden, "A large-scale study of usability criteria addressed by static analysis tools," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022, pp. 532–543.

[49] A. S. Ami, K. Moran, D. Poshyvanyk, and A. Nadkarni, "" false negative-that one is going to kill you": Understanding industry perspectives of static analysis based security testing," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 3979–3997.

[50] S. Lipp, S. Banescu, and A. Pretschner, "An empirical study on the effectiveness of static c code analyzers for vulnerability detection," in *Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis*, 2022, pp. 544–555.

[51] S. Cao, X. Sun, L. Bo, R. Wu, B. Li, and C. Tao, "Mvd: memory-related vulnerability detection based on flow-sensitive graph neural networks," in *Proceedings of the 44th international conference on software engineering*, 2022, pp. 1456–1468.

[52] Y. Xiao, B. Chen, C. Yu, Z. Xu, Z. Yuan, F. Li, B. Liu, Y. Liu, W. Huo, W. Zou *et al.*, "{MVP}: Detecting vulnerabilities using {Patch-Enhanced} vulnerability signatures," in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 1165–1182.

[53] D. Zou, S. Wang, S. Xu, Z. Li, and H. Jin, "μ vuldeepecker: A deep learning-based system for multiclass vulnerability detection," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 5, pp. 2224–2236, 2019.

[54] W. Wang, T. N. Nguyen, S. Wang, Y. Li, J. Zhang, and A. Yadavally, "Deepvd: Toward class-separation features for neural network vulnerability detection," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2249–2261.

[55] S. Cao, X. Sun, L. Bo, R. Wu, B. Li, X. Wu, C. Tao, T. Zhang, and W. Liu, "Learning to detect memory-related vulnerabilities," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 2, pp. 1–35, 2023.

[56] W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Unified pre-training for program understanding and generation," *arXiv preprint arXiv:2103.06333*, 2021.

[57] Y. Jiang, Y. Zhang, X. Su, C. Treude, and T. Wang, "Stagedvulbert: Multi-granular vulnerability detection with a novel pre-trained code model," *IEEE Transactions on Software Engineering*, 2024.

[58] J. Zhang, Z. Liu, X. Hu, X. Xia, and S. Li, "Vulnerability detection by learning from syntax-based execution paths of code," *IEEE Transactions on Software Engineering*, vol. 49, no. 8, pp. 4196–4212, 2023.

[59] H. Wang, Z. Tang, S. H. Tan, J. Wang, Y. Liu, H. Fang, C. Xia, and Z. Wang, "Combining structured static code information and dynamic symbolic traces for software vulnerability prediction," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.

[60] Y. Chen, Z. Ding, L. Alowain, X. Chen, and D. Wagner, "Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection," in *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*, 2023, pp. 654–668.

[61] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, "Vulnerability detection with code language models: How far are we?" *arXiv preprint arXiv:2403.18624*, 2024.